

# Автоматизация разработки

## ПО лекция 2

### Git и его анатомия

# История версионирования



git - мерзавец

# 2005: рождение Git

## Git: новый подход к версионированию

Создан как ответ на реальные проблемы, чтобы сделать разработку больших проектов проще, быстрее и надёжнее.

### 1 Проблема

Конец эпохи BitKeeper

- ✗ В 2005 году разработчики Linux потеряли бесплатный доступ к **BitKeeper**
- ✗ BitKeeper был коммерческим продуктом с закрытым исходным кодом
- ✗ Существующие альтернативы (CVS, SVN, Mercurial и др.) не удовлетворяли требованиям к скорости и удобству
- ✗ Нужна была система, которой можно доверить развитие огромного проекта, как **Linux Kernel**

### 2 Решение

Линус Торвалдс



*"Нам нужна простая, быстрая, надёжная и полностью распределённая система контроля версий."*  
— Линус Торвалдс, 2005 год

В 2005 году Линус Торвалдс начал разработку новой системы, которая изначально создавалась для разработки **Linux Kernel** и сообщества, а не как коммерческий продукт.



Главная цель — дать разработчикам полный контроль над историей проекта без зависимости от внешнего владельца.

### 3 Что такое Git?

Распределённая система контроля версий



- ✓ У каждого разработчика есть полная история проекта
- ✓ Работает быстро, даже с большими кодовыми базами
- ✓ Поддерживает эффективную параллельную разработку
- ✓ Независим от конкретного сервера или сервиса
- ✓ Спроектирован с учётом реальных задач большого open source проекта

### 4 Ключевые идеи



#### Скорость

Все основные операции (commit, branch, merge) работают очень быстро, даже на больших проектах.



#### Распределённость

Каждый разработчик имеет полноценный репозиторий с полной историей.



#### Снимки проекта

Git хранит не отличия файлов, а состояния всего проекта (snapshot), используя эффективное хранение объектов.



#### Работа с ветками

Ветки лёгкие и дешёвые — это просто указатели на коммиты, а не копии проекта.



#### Надёжность

Контроль целостности через хеширование, защита от потери данных.



#### Создан для сообщества

Открытый исходный код и активное развитие.

### 5 Результат

- **2005** Начало разработки новой системы
- **2005** Первые версии и использование в разработке Linux Kernel
- **2007** Git становится зрелым и удобным для широкого круга проектов
- **2008** Появляется GitHub — платформа для совместной работы вокруг Git
- **Сегодня** Git — стандарт de facto в индустрии и open source



#### Git изменил разработку

Он дал миллионам разработчиков по всему миру простой, мощный и свободный инструмент для совместной работы.

“Git был создан, потому что нам нужна была лучшая система, и мы могли её сделать.”

— Линус Торвалдс

# Магия git init

## Что делает git init?

Как обычная папка превращается в локальный Git-репозиторий.

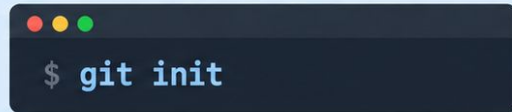
### До git init

Обычная папка с файлами проекта.



### Запускаем команду

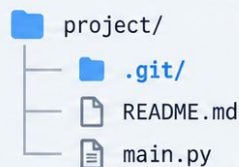
В терминале, находясь в папке проекта.



Git создаёт скрытую папку `.git/` и инициализирует локальный репозиторий.

### После git init

В папке появляется скрытая папка `.git/`.



### Что находится внутри `.git/`?

Основные файлы и папки (упрощённо).



#### HEAD

указывает, где мы сейчас находимся.



#### config

настройки репозитория.



#### objects/

Git-объекты: blobs, trees, commits.



#### refs/

ветки и ссылки на commits.



#### hooks/

скрипты, которые Git может запускать.



#### index

staging area; появляется и обновляется при git add.



#### Важно:

1

git init не создаёт commit автоматически.

2

git init не отправляет ничего в интернет.

3

После git init история ещё пуста — она появится после git add и git commit.



#### Вывод:

`.git/` — это локальная база данных Git. Файлы проекта рядом — это рабочая копия.

# git add [ blob - Binary Large Object ]



## Что происходит внутри .git?



### Вывод:

git add создаёт/обновляет blob, записывает его hash в index и подготавливает выбранное состояние файла к следующему commit.

# git status

## Что делает git status?

Как Git показывает текущее состояние репозитория и что именно он сравнивает.



## Что именно показывает git status?



## Как Git это определяет?



## Жизненный цикл файлов



- Важно:**
- `git status` ничего не изменяет файлы и не создаёт commit.
  - Это первая команда, если непонятно, что происходит.
  - Она сравнивает HEAD, index и рабочую папку.
- Вывод:** `git status` — это команда для проверки текущего состояния репозитория. Она показывает, что уже подготовлено, что изменено и что ещё не отслеживается, сравнивая HEAD, index и рабочую папку.

# .gitignore

## 1 Что такое .gitignore

.gitignore — это текстовый файл, в котором мы указываем, какие файлы и папки не нужно отслеживать в Git.

- ✓ Файлы не будут показаны как untracked
- ✓ Их нельзя случайно добавить в репозиторий
- ✓ Обычно .gitignore коммитится в репозиторий
- ✓ Это обычный текстовый файл с правилами

## 5 Уже отслеживаемые файлы

.gitignore работает только для файлов, которые ещё не добавлены в репозиторий.

- ❗ Если файл уже был добавлен, его нужно сначала удалить из индекса.

```
1 # файл уже в репозитории
2 git rm --cached .env
3 git commit -m "Stop tracking .env"
```

## 2 Пример .gitignore

Типичный .gitignore для Python-проекта:

```
1 # Окружение
2 .env
3 .venv/
4
5 # Кэш и временные файлы
6 __pycache__/
7 *.pyc
8 *.pyo
9
10 # Логи
11 *.log
12
13 # Сборка
14 build/
15 dist/
16
17 # Системные файлы
18 .DS_Store
```

Файл останется на диске, но Git больше не будет его отслеживать.

## 3 Как это работает

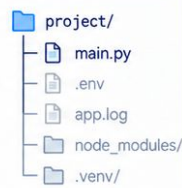
Git игнорирует файлы, которые совпадают с правилами из .gitignore.

### Без .gitignore



Все эти файлы будут показаны как untracked

### С .gitignore



Эти файлы игнорируются и не отображаются в git status

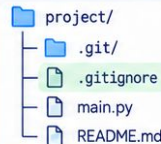
## 4 Полезные паттерны

Часто используемые правила:

|                |                              |
|----------------|------------------------------|
| file.txt       | Игнорировать конкретный файл |
| folder/        | Игнорировать папку           |
| *.log          | Все файлы с расширением .log |
| build/         | Всё содержимое папки         |
| .DS_Store      | Системные файлы (macOS)      |
| *.pyc          | Все скомпилированные файлы   |
| !important.log | Исключение (не игнорировать) |

## 6 Где находится

.gitignore лежит в корне репозитория (или в любой подпапке).



💡 Можно использовать несколько .gitignore в разных папках проекта.



### Главная мысль

.gitignore не удаляет файлы и не прячет историю. Он только говорит Git, какие неотслеживаемые файлы не показывать и не добавлять в репозиторий.

# git commit

## 1. До git commit



```
.git/index  
README.md -> blob A  
src/main.py -> blob B  
...
```



```
.git/objects/  
a1b2c3... (blob A)  
d4e5f6... (blob B)  
...
```

После git add содержимое файлов уже подготовлено в index.

## 2. Что делает git commit

```
$ git commit -m "Add hello"
```



```
index  
README.md -> blob A  
src/main.py -> blob B  
...
```



```
tree  
92a7c...
```



```
commit  
9f8e7d...
```

Git строит tree по index и создаёт новый commit object.

## 3. После git commit



Текущая ветка сдвигается на новый commit.

## Как Git работает с файлами

```
project/  
├-- README.md  
└-- src/  
    └-- main.py
```

### ROOT TREE

```
├-- README.md -> blob A  
└-- src -> TREE B  
    └-- main.py -> blob C
```

**Blob** хранит содержимое файла.  
**Tree** хранит структуру каталога: имена, типы и ссылки на объекты.

## Что находится внутри commit

```
tree 92a7c...  
parent 43bd1...  
author Artem <artem@example.com> 1789410000 +0300  
committer Artem <artem@example.com> 1789410000 +0300  
.....  
Add hello
```

- tree** — ссылка на корневой tree
- parent** — ссылка на родительский commit
- author** — автор коммита
- committer** — кто создал commit
- date** — дата и время (Unix timestamp)
- message** — текст сообщения

Commit хранит ссылку на корневой tree и метаданные, а не сами файлы.

## Как считается hash commit

```
Содержимое commit -> "commit <size>\0<content>" -> # SHA-1 -> 9f8e7d...
```

Пример формулы:

```
commit_hash = SHA1("commit " + size + "\0" + commit_content)
```

- Изменился файл -> меняется blob -> tree -> commit hash
- Изменилось сообщение commit -> другой hash
- Изменилось время или parent -> другой hash

1 git commit берёт данные из index, а не напрямую из рабочей папки.

2 git commit не отправляет изменения на GitHub.

3 После commit новый hash записывается в refs/heads/main.



**Вывод:** git commit превращает состояние index в tree, создаёт commit object, вычисляет его hash и передвигает текущую ветку на новый commit.

# Интерактив 1

## 1 Создаём папку проекта

```
$ mkdir git-internals-demo
$ cd git-internals-demo
$ ls -la
```

```
$ ls -la
total 8
drwxr-xr-x  2 user user 4096 Mar  8 12:00 .
drwxr-xr-x 10 user user 4096 Mar  8 12:00 ..
```

Пока это обычная пустая директория, без какой-либо Git-структуры.

## 2 Запускаем Git

```
$ git init --object-format=sha1
$ git branch -M main
```

Initialized empty Git repository in ../.git/

Git создаёт скрытую папку .git/ и превращает директорию в локальный Git-репозиторий.

## 3 Смотрим, что появилось

```
$ ls -la
$ ls -la .git
```

```
git-internals-demo/
└── .git/
    ├── HEAD
    ├── config
    ├── description
    ├── hooks/
    ├── info/
    ├── objects/
    └── refs/
```

Теперь появилась папка .git/. В ней хранится вся внутренняя структура Git.

## 4 Смотрим HEAD и проверяем ветку

```
$ cat .git/HEAD
ref: refs/heads/main
```

```
$ cat .git/refs/heads/main
cat: .git/refs/heads/main: No such file or directory
```

```
$ git status

On branch main
No commits yet
nothing to commit (create/copy files and use "git add" to track)
```

HEAD уже указывает на ветку main, но файла ветки пока нет, потому что ещё не создан первый commit.

## Что изменилось?

### До git init

git-internals-demo/  
обычная папка

- нет папки .git/
- история отсутствует
- это просто директория

### После git init

git-internals-demo/  
└── .git/  
локальный Git-репозиторий

- появилась скрытая папка .git/
- история пока пустая
- файлы проекта не изменились

## Как устроена ссылка HEAD?

### HEAD

↓  
refs/heads/main  
↓  
???

### Сейчас (после git init)

HEAD → main → ???  
ветка main существует, но файла ещё нет

### После первого commit

HEAD → main → <commit-hash>  
после первого commit в файле ветки появится хеш коммита



## Если интересно дополнительно

```
find .git -maxdepth 3 -print | sort
```

покажет начальную структуру .git

```
cat .git/HEAD
```

покажет, на какую ветку смотрит HEAD

```
git status
```

подтвердит, что commit ещё нет



**Вывод:** git init создаёт папку .git/ и инициализирует локальный репозиторий, но не создаёт commit и не отправляет ничего в интернет. HEAD уже существует и указывает на ветку main, первый commit пока не создан.

# Интерактив 2

## 1 Добавляем файл в staging area

```
$ git add hello.txt
$ git status
$ ls -l .git/index
$ git ls-files --stage
$ find .git/objects -type f
```

### Результат (упрощённо):

Changes to be committed:  
new file: hello.txt

.git/index появился:  
-rw-r--r-- 1 user user 137 Nov 10 12:00 .git/index

git ls-files --stage:  
100644 b6fc4c620b67d95f953a5c1c1230aaab5db5a1b0 0 hello.txt

Найденный объект в .git/objects:  
.git/objects/b6/fc4c620b67d95f953a5c1c1230aaab5db5a1b0

Теперь Git знает, какой blob соответствует hello.txt.

## 2 Что изменилось внутри .git?

### До git add

#### Working Directory:

hello.txt  
(содержит "hello")

#### .git/index:

нет

#### .git/objects:

пусто

### После git add

#### Working Directory:

hello.txt  
(содержит "hello")

#### .git/index:

hello.txt ->  
b6fc4c...

#### .git/objects:

blob b6fc4c...

### Как выглядит hash внутри .git/objects:

b6fc4c620b67... → .git/objects/b6/  
fc4c620b67...

Hash разбивается на первые 2 символа (папка), остальные 38 символов (имя файла).

## 3 Меняем файл после git add

```
$ printf "hello world" > hello.txt
$ git status
```

### Результат (упрощённо):

Changes to be committed:  
new file: hello.txt

Changes not staged for commit:  
(use "git add <file>..." to update what will be committed)  
modified: hello.txt

### Сравнение версий файла:

#### Index (staging area):

hello.txt = "hello"

#### Working Directory:

hello.txt =  
"hello world"

После git add можно снова изменить файл. В staging area останется старая версия.

## 4 Как это увидеть

```
$ git diff # Index ↔ Working Directory
$ git diff --staged # HEAD ↔ Index
```

В этом примере staged diff показывает, что Git собирается добавить в первый commit версию "hello".

### Сравнение, которое показывает каждая команда:

#### git diff

(Index ↔ Working Directory)

```
--- a/hello.txt
+++ b/hello.txt
-hello
+hello world
```

Показывает изменения между staging area и текущей версией файла.

#### git diff --staged

(HEAD ↔ Index)

```
--- /dev/null
+++ b/hello.txt
+hello
```

Показывает, что Git собирается добавить в первый commit: новый файл hello.txt со строкой hello.

## Ключевая мысль



Working Directory  
(ваши файлы)

git add



Index / Staging Area  
(зафиксированная версия)

git commit



Repository  
(история коммитов)

git add фиксирует конкретное состояние файла для следующего commit.



## Если интересно дополнительно

```
git cat-file -t <hash>
git cat-file -s <hash>
git cat-file -p <hash>
git status
```

- покажет тип объекта
- покажет размер объекта
- покажет содержимое blob
- покажет staged и modified одновременно



**Вывод:** git add создаёт/обновляет blob и записывает его hash в .git/index. Если потом изменить файл ещё раз, новая версия не попадёт в commit автоматически — нужно снова выполнить git add.

# Интерактив 3

## 1 Готовимся к commit

```
printf "hello" > hello.txt
git config user.name "Student"
git config user.email "student@example.com"
```

Возвращаем hello.txt к staged-версии и настраиваем автора commit.

```
.git/config
```

```
[user]
  name = Student
  email = student@example.com
```

## 2 Создаём первый commit

```
git commit -m "Add hello"
```

```
[main (root-commit) 8f3307f] Add hello
1 file changed, 1 insertion(+)
create mode 100644 hello.txt
```

Git строит tree по index и создаёт новый commit object.

## 3 Что изменилось в .git

.git/objects (после commit)

```
blob    b6fc4c...
tree    04df07...
commit  8f3307...
```

index → tree → commit

До commit был только blob.  
После commit появились tree и commit.

## 4 Ветка main теперь указывает на commit

```
cat .git/HEAD
ref: refs/heads/main
```

```
cat .git/refs/heads/main
8f3307f14a5dfa63258856f1dc0ad14f218d12b4
```

```
git status
On branch main
nothing to commit, working tree clean
```

HEAD → main → 8f3307...

## 💡 Ключевая мысль



git commit берёт данные из index, а не напрямую из рабочей папки.

## ⚙️ Если интересно дополнительно

|                              |                                |
|------------------------------|--------------------------------|
| find .git/objects -type f    | — покажет объекты после commit |
| git log --oneline --decorate | — покажет HEAD и main          |
| git cat-file -p HEAD         | — покажет содержимое commit    |

✅ **Вывод:** git commit превращает состояние index в tree, создаёт commit object и передвигает текущую ветку main на новый commit.

# Интерактив 4

## 1 Смотрим содержимое commit

```
$ git cat-file -p HEAD
```

```
tree 04df07b08ca746b3167d0f1d1514e2f39a52c16c
author Student <student@example.com> 1789412134 +0300
committer Student <student@example.com> 1789412134 +0300

Add hello
```

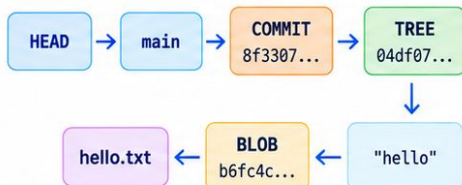
Commit хранит ссылку на tree и метаданные.

## 2 Переходим к tree и blob

```
$ git cat-file -p HEAD^{tree}
```

```
100644 blob b6fc4c620b67d95f953a5c1c1230aaab5db5a1b0 hello.txt
```

```
$ git cat-file -p b6fc4c620b67d95f953a5c1c1230aaab5db5a1b0
hello
```



## 3 Получаем raw commit

```
$ git cat-file commit HEAD > commit.raw
```

```
$ cat commit.raw
```

```
tree 04df07b08ca746b3167d0f1d1514e2f39a52c16c
author Student <student@example.com> 1789412134 +0300
committer Student <student@example.com> 1789412134 +0300

Add hello
```

```
$ wc -c < commit.raw
168
```

Размер commit.raw = 168 байт.

## 4 Считаем hash commit руками

```
$ SIZE=$(wc -c < commit.raw | tr -d ' ')
```

```
$ { printf "commit %s\0" "$SIZE";
  cat commit.raw; } | shasum -a 1
```

```
$ git rev-parse HEAD
```

```
8f3307f14a5dfa63258856f1dc0ad14f218d12b4
```



```
8f3307f14a5dfa63258856f1dc0ad14f218d12b4
```

```
SHA1( "commit 168\0" + <commit_content> )
```

## Что важно понять

- **commit hash** не случайный — он вычисляется из содержимого commit
- Изменится **message** → изменится hash
- Изменится **author, time или parent** → изменится hash
- Изменится **tree** → изменится и hash commit

## Если интересно дополнительно

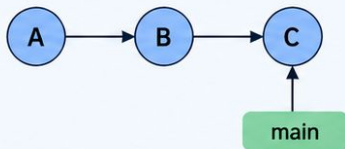
- |                              |                         |
|------------------------------|-------------------------|
| git cat-file -t HEAD         | — покажет тип объекта   |
| git cat-file -s HEAD         | — покажет размер commit |
| git rev-parse HEAD^{tree}    | — выведет hash tree     |
| git log --oneline --decorate | — покажет HEAD и main   |

✓ **Вывод:** commit хранит ссылку на tree и метаданные, а его hash вычисляется как SHA-1 от строки вида "commit <size>\0<content>".

# git branch

## 1 История коммитов

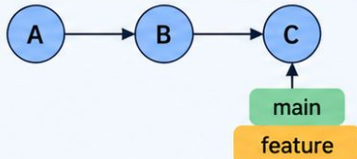
У нас есть последовательность коммитов.



## 2 Создаём новую ветку

Новая ветка — это ещё одно имя на тот же commit. Никакие файлы не копируются.

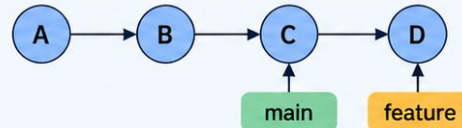
```
git branch feature
```



## 3 Работаем в новой ветке

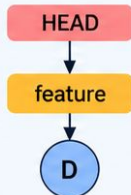
Переключаемся на ветку и делаем новые коммиты. Ветка автоматически сдвигается вперёд.

```
git switch feature  
git commit -m "Add feature"
```



## 4 Что такое HEAD

HEAD — это указатель на текущую ветку (а значит, на её последний commit).



## 5 Как это хранится в репозитории

Ветки — это обычные файлы с хешами коммитов.

```
.git/  
├── HEAD      → refs/heads/feature  
└── refs/heads/  
    ├── main   → <hash C>  
    └── feature → <hash D>
```

## 6 Главные выводы

1

Ветка — это подвижный указатель на commit.

2

Создание ветки не копирует проект.

3

Несколько веток могут указывать на один и тот же commit.

4

При новом коммите ветка автоматически сдвигается вперёд.

5

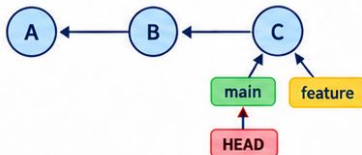
HEAD указывает на текущую ветку.

# git branch / git switch / git checkout

## 1 Создаём ветку

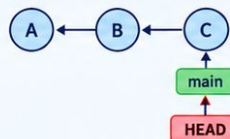
```
git branch feature
```

Команда создаёт новую ветку, но не переключает на неё.



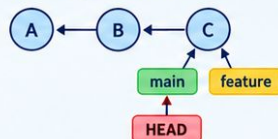
## 2 Что изменилось?

До git branch



- Появилась новая ссылка feature
- HEAD не изменился
- Файлы проекта не копируются

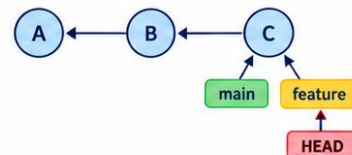
После git branch



## 3 Переключаемся

```
git switch feature
```

Теперь HEAD смотрит на ветку feature.



## 4 Как меняется HEAD

До git switch:

```
.git/HEAD  
ref: refs/heads/main
```

После git switch:

```
.git/HEAD  
ref: refs/heads/feature
```

git switch переписывает .git/HEAD, меняя текущую ветку.

## 5 Новый commit двигает только текущую ветку

```
git commit -m "Add feature"
```



После commit движется только активная ветка.

## 6 git checkout — старая универсальная команда

```
git checkout feature
```

Переключается на ветку (как git switch feature).

```
git checkout -b feature
```

Создаёт ветку и сразу переключается на неё.

```
git checkout <commit>
```

Переходит к конкретному commit (может привести к detached HEAD).

```
git checkout -- file.txt
```

Старая команда для файлов. Теперь понятнее: git restore file.txt.

## Главная мысль

1

git branch создаёт ветку

2

git switch меняет HEAD

3

HEAD указывает на текущую ветку

4

ветка — это ссылка на commit

5

новый commit двигает только текущую ветку

6

git checkout — старая универсальная команда, а switch и restore — более понятные современные замены



Если интересно дополнительно

```
git branch  
git branch -a  
cat .git/HEAD  
cat .git/refs/heads/main
```

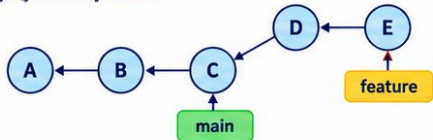
- список веток
- все ветки
- куда смотрит HEAD
- на какой commit указывает ветка

# git merge --squash

## 1 merge идёт в текущую ветку

```
git switch main  
git merge feature
```

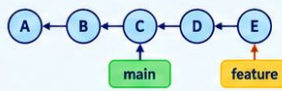
Git берёт изменения из feature и добавляет их в текущую ветку main.



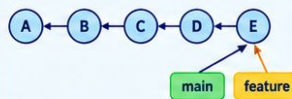
## 2 Fast-forward

Если main не менялась после отвлечения, Git просто передвигает указатель main вперёд. Новый commit не создаётся.

До merge

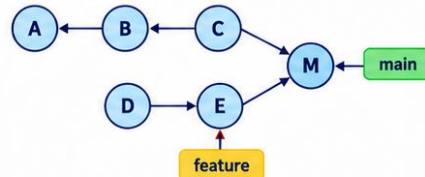


После merge (fast-forward)



## 3 Merge commit

Если ветки развивались отдельно, Git создаёт новый merge commit. У merge commit два parent.



## 4 Обычный merge и --no-ff

```
git merge feature
```

Обычный merge: Git сам выберет fast-forward или merge commit.

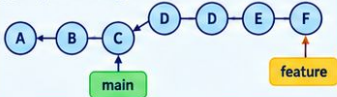
```
git merge --no-ff feature
```

--no-ff: даже если можно передвинуть main вперёд, будет создан отдельный merge commit.

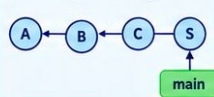
## 5 Squash merge

```
git merge --squash feature  
git commit -m "Add feature"
```

До squash merge



После squash merge



Все изменения ветки собираются в один новый commit.

## 6 Чем squash отличается

| Обычный merge                 | Squash merge                                     |
|-------------------------------|--------------------------------------------------|
| сохраняет историю ветки       | объединяет изменения в один commit               |
| может создать merge commit    | история ветки как отдельная линия не переносится |
| в графе видно отдельную линию | граф истории получается чище                     |



### Главная мысль

1

merge работает в текущую ветку

2

fast-forward только двигает указатель

3

merge commit соединяет две линии истории

4

--no-ff сохраняет явный merge

5

--squash собирает ветку в один commit



### Если интересно дополнительно

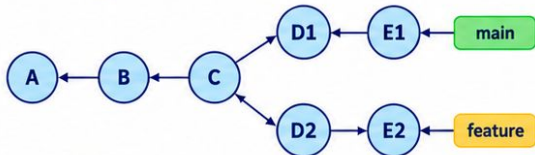
```
git log --oneline --graph --all  
git merge --no-ff feature  
git merge --squash feature
```

— покажет граф истории  
— всегда создаст merge commit  
— подготовит один commit из всей ветки

# Конфликты

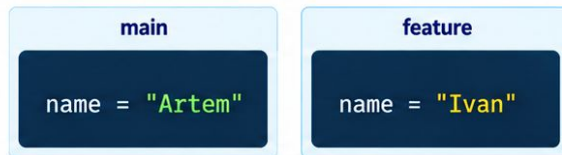
## 1 Когда появляется конфликт

Конфликт возникает, когда Git не может автоматически выбрать итоговый результат после merge.



## 2 Обе ветки меняют одно и то же место

Изменена одна и та же строка по-разному.



## 3 Что происходит при merge

```
git switch main
git merge feature
```

В результате Git показывает конфликт в файле:

```
<<<<<< HEAD
name = "Artem"
=====
name = "Ivan"
>>>>>> feature
```

Верхняя часть — текущая ветка (в данном случае main), нижняя — вливаемая ветка (feature).

## 4 Как решить конфликт



Пример итогового варианта:

```
name = "Artem Ivan"
```

Можно выбрать версию main, feature или сделать третий вариант.

## 5 Какие команды выполнить

```
git status
git add file.txt
git commit
```

- `git status` — покажет конфликтующие файлы
- `git add` — пометит конфликт как решённый
- `git commit` — завершит merge

## 6 Главная мысль

- 1 конфликт — это нормальная ситуация
- 2 Git умеет объединять текст, но не принимает смысловое решение за человека
- 3 конфликт решается вручную
- 4 после исправления нужно сделать `git add` и `git commit`



## Вывод:

Конфликт возникает, когда две ветки по-разному изменили одно и то же место. Git останавливает merge, показывает оба варианта и ждёт, пока человек выберет итоговый результат.

# git remote

## 1 Что такое remote

Remote в Git — это сохранённая именованная ссылка на другой Git-репозиторий. Этот репозиторий может находиться на сервере, другом компьютере или в другой папке на этом же компьютере.



```
git remote add origin /tmp/other-repo.git
git remote add origin <URL>
```

## 2 origin — это просто имя

**origin** не является специальной сущностью Git. Это просто общепринятое имя для первого remote. Вы можете использовать любые другие имена.

```
git remote add server <URL>
git remote add backup <URL>
```

| Имя remote | Указывает на |
|------------|--------------|
| origin     | репо A       |
| server     | репо A       |
| backup     | репо B       |

! Вы можете выбрать любое имя для remote.

## 3 Где это хранится

После выполнения команды `git remote add origin <URL>` Git обновляет файл `.git/config`.

`.git/config`

```
[remote "origin"]
  url = <URL>
  fetch = +refs/heads/*:refs/remotes/origin/*
```

В этом файле хранятся все настроенные remote и их параметры (адреса для fetch и push).

## 4 Как посмотреть

Список настроенных remote и их адреса:

```
git remote -v
origin /tmp/other-repo.git (fetch)
origin /tmp/other-repo.git (push)
```

➔ Показывает все настроенные remote и адреса для fetch (скачивание) и push (отправка).

Подробная информация о remote:

```
git remote show origin
```

➔ Показывает подробную информацию о конкретном remote (например, origin).

## 5 Локальные и remote-tracking ветки



origin/main — это не сама удалённая ветка, а локальная ссылка на последнее известное состояние удалённой ветки.

! Ваша локальная ветка main может быть впереди origin/main.

## 6 Что важно понять

- 1 Remote — это именованная ссылка на другой Git-репозиторий.
- 2 origin — просто общепринятое имя.
- 3 Remote сам по себе не синхронизирует ничего автоматически.
- 4 Обмен данными выполняют команды fetch, push и pull.



## Вывод

Remote — это именованная ссылка на другой Git-репозиторий. **origin** — просто стандартное имя такого remote, а не специальный сервер или функция Git. Сам по себе remote ничего не синхронизирует: обмен выполняют команды fetch, push и pull.

# Поднимаемся выше

## 1 GitHub

Самая популярная платформа для хостинга Git-репозиторий и совместной разработки.



- ✓ Огромное open-source сообщество
- ✓ Pull Request (PR)
- ✓ Issues, Projects, Wiki
- ✓ GitHub Actions (CI/CD)
- ✓ Packages, Codespaces
- ✓ Хорошая интеграция с внешними инструментами



Лучший выбор для open source и публичных проектов.

## 2 GitLab

Полноценная DevOps-платформа с открытым решением и self-hosted версией.



- ✓ Merge Request (MR)
- ✓ Встроенный CI/CD
- ✓ DevSecOps (SAST, DAST, Dependency Scanning)
- ✓ Container Registry
- ✓ Управление задачами
- ✓ Можно развернуть в своей инфраструктуре



Хороший выбор для компаний, которые хотят единое решение и контроль над инфраструктурой.

## 3 Bitbucket

Платформа от Atlassian, тесно интегрированная с Jira и Confluence.



- ✓ Pull Request (PR)
- ✓ Bitbucket Pipelines (CI/CD)
- ✓ Интеграция с Jira, Confluence
- ✓ Удобен для корпоративной разработки
- ✓ Поддержка частных репозиторий
- ✓ Хорош для команд, использующих экосистему Atlassian



Оптимален, если в компании уже используются Jira и другие продукты Atlassian.

## 4 Azure DevOps (Azure Repos)

Решение от Microsoft для enterprise-разработки.



- ✓ Azure Repos (Git)
- ✓ Pull Request (PR)
- ✓ Azure Pipelines (CI/CD)
- ✓ Boards (управление задачами)
- ✓ Artifacts (хранилище артефактов)
- ✓ Интеграция с экосистемой Microsoft (Azure, Visual Studio)



Подходит для корпоративных команд и проектов в экосистеме Microsoft.

## 5 Gitee

Одна из крупнейших китайских платформ для разработки.



- ✓ Популярна в Китае
- ✓ Pull Request (PR)
- ✓ Issues, Wiki, DevOps
- ✓ Быстрая работа в китайском регионе
- ✓ Поддержка частных репозиторий

## 6 GitCode / AtomGit

Китайская платформа, связанная с CSDN. Развивается как инфраструктура для разработчиков и open source.



- ✓ Pull Request (PR)
- ✓ CI/CD и DevOps инструменты
- ✓ Интеграция с экосистемой CSDN
- ✓ Подходит для китайского рынка

## 7 CODING (Tencent Cloud)

Корпоративная DevOps-платформа от Tencent.



- ✓ Код-хостинг (Git/SVN)
- ✓ CI/CD, тестирование и деплой
- ✓ Управление проектами
- ✓ Подходит для больших команд и enterprise-проектов
- ✓ Популярен в Китае

## 8 Другие популярные



**SourceForge** Крупный каталог open source проектов



**Codeberg** Некоммерческая альтернатива (на базе Forgejo)



**SourceHut** Минималистичный сервис с акцентом на open source



**AWS CodeCommit** Git-хостинг в экосистеме AWS



**GitKraken** Инструменты для работы с Git (клиент, не хостинг)



### Главная мысль

GitHub, GitLab, Bitbucket, Gitee и другие — это не **Git**, а **сервисы вокруг Git**. Они предоставляют хостинг репозиторий, инструменты для совместной разработки, CI/CD, управление задачами и права доступа. Если вы понимаете Git, перейти между этими платформами обычно несложно — меняется интерфейс, но базовые команды и принципы остаются теми же.

# Возвышаемся

## 1 Российские аналоги: зачем они нужны?

Российские Git-платформы — это альтернативы GitHub и GitLab, которые развиваются в России и подходят для корпоративного и государственного сектора.

- ✓ Соответствие требованиям регуляторов
- ✓ Размещение данных в РФ
- ✓ Интеграция с отечественной инфраструктурой
- ✓ Поддержка и развитие локальными вендорами
- ✓ Полный набор DevOps-инструментов

**i** Это не замена Git. Везде используется тот же Git, меняется только платформа вокруг него.



## 2 GitVerse (СберТех)

Российская платформа от Сбера для разработки и совместной работы.



- ✓ Git-репозитории (публичные и частные)
- ✓ Pull / Merge Request, code review
- ✓ Issues, Wiki, проекты
- ✓ CI/CD (свои и облачные runners)
- ✓ Package Registry, Pages
- ✓ Интеграция с GigaCode и GigaIDE
- ✓ Размещение в РФ, соответствие требованиям

```
# Клонирование репозитория
git clone https://gitverse.ru/group/project.git

# Отправка изменений
git push origin main
```

## 3 GitFlic (ГК «Астра»)

Российская платформа, сейчас развивается в экосистеме ГК «Астра».



- ✓ Git-репозитории и code review
- ✓ Полный CI/CD
- ✓ Container и package registry (Docker, Maven, npm, PyPI, NuGet и др.)
- ✓ Инструменты безопасности (SAST, dependency scanning)
- ✓ SSO/LDAP/SAML, гибкое управление правами
- ✓ Есть SaaS и self-hosted (Astra Linux, Docker, K8s)

```
# Клонирование репозитория
git clone https://gitflic.ru/group/project.git

# Создание merge request
# (через веб-интерфейс или CLI)
```

## 4 SourceCraft (Яндекс)

Платформа для разработки от Яндекса с интеграцией в экосистему Yandex Cloud.



- ✓ Git-репозитории, code review
- ✓ CI/CD (файл `.sourcecraft/ci.yaml`)
- ✓ Управление задачами и проектами
- ✓ Интеграция с Yandex Cloud
- ✓ AI-инструменты (SourceCraft, YandexGPT)
- ✓ Поддержка GitHub Actions и GitLab pipelines
- ✓ Размещение в РФ

```
# Пример CI/CD конфигурации
# .sourcecraft/ci.yaml
version: 1
stages: 1
- build
- test
- deploy
```

## 5 Краткое сравнение

| Платформа   | Разработчик | Основной фокус            | CI/CD | Registry | AI / Доп. возможности    | Self-hosted | Подходит для                             |
|-------------|-------------|---------------------------|-------|----------|--------------------------|-------------|------------------------------------------|
| GitVerse    | СберТех     | Универсальная платформа   | ✓     | ✓        | GigaCode, GigaIDE        | ✓           | Команды и компании любого масштаба       |
| GitFlic     | ГК «Астра»  | DevOps и безопасность     | ✓     | ✓        | Инструменты безопасности | ✓           | Корпоративный сектор, госорганизации     |
| SourceCraft | Яндекс      | Интеграция с облаком и AI | ✓     | ✓        | Yandex Cloud, YandexGPT  | ✓           | Команды, использующие экосистему Яндекса |

### Главная мысль

GitVerse, GitFlic, SourceCraft и другие российские платформы — это полноценные аналоги GitHub/GitLab, построенные вокруг того же Git. Они предоставляют хостинг репозитория, инструменты для совместной разработки, CI/CD и интеграции с отечественной инфраструктурой.

# GitHub

## 1 Что такое GitHub

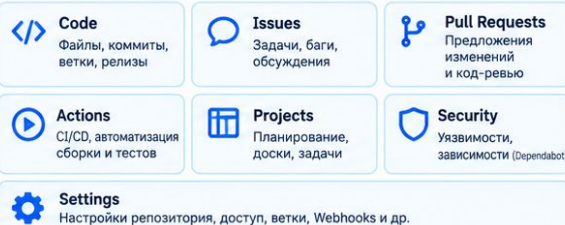
GitHub — это платформа вокруг Git, а не замена Git.



- Удалённые репозитории (в облаке).
- Совместная работа над кодом.
- Код-ревью через Pull Requests.
- Автоматизация и CI/CD (GitHub Actions).

## 2 Что есть в репозитории GitHub

Репозиторий на GitHub — это Git-репозиторий + возможности платформы.



## 3 Как создать репозиторий

Пошаговая инструкция:



Совет: дайте понятное имя и, при необходимости, добавьте описание.

## 4 Если локальный repo уже есть

Если у вас уже есть локальный репозиторий на компьютере, лучше создать пустой репозиторий на GitHub.

Не добавляйте README / .gitignore / LICENSE, если потом будете пушить уже существующий локальный repo.



## 5 Связываем локальный repo с GitHub

Добавляем удалённый репозиторий и проверяем настройку:

```
$ git remote add origin https://github.com/user/git-demo.git
$ git remote -v
```

Фрагмент файла .git/config:

```
[remote "origin"]
url = https://github.com/user/git-demo.git
fetch = +refs/heads/*:refs/remotes/origin/*
```

origin = просто имя remote.

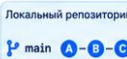
## 6 Первый push

Отправляем локальные коммиты на GitHub:

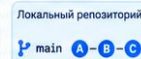
```
$ git push -u origin main
```

origin = куда (удалённый репозиторий)    main = какая ветка (что отправляем)    -u = настроить upstream (запомнить связь)

До push:



После push:



## Главная мысль

GitHub — это сервис вокруг Git: удалённые репозитории, PR, review, Issues, Actions и другие инструменты. Сам Git остаётся тем же: clone, add, commit, branch, merge, push.

# git clone

## 1 Команда git clone

Клонирование — это создание полной локальной копии удалённого репозитория с историей, ветками и настройками.

```
git clone https://github.com/user/project.git
# можно указать свою папку
git clone https://github.com/user/project.git my-project
```

- ✓ Создаётся локальный Git-репозиторий
- ✓ Загружается вся история (commits, trees, blobs)
- ✓ Настраивается remote origin
- ✓ Создаётся локальная ветка (обычно main)
- ✓ Файлы разворачиваются в рабочую папку

## 2 Запрос к удалённому репозиторию

Git обращается к удалённому репозиторию, запрашивает список ссылок (refs) и метаданные.



Удалённый репозиторий сообщает, какие ветки существуют и на какие коммиты они указывают.

```
main → a1b2c3d
develop → f4e5d6c
v1.0 → tag: 9f8e7d
```

## 3 Загрузка объектов

Git скачивает необходимые объекты (коммиты, деревья, blobs). Обычно они передаются в сжатом виде (packfile).

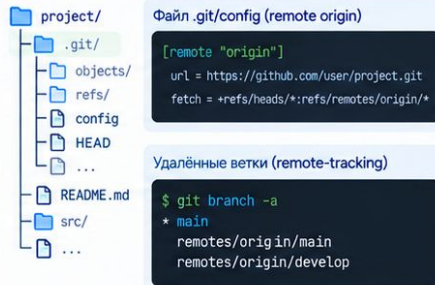


Какие объекты загружаются:

- Commits история изменений
- Trees структура каталогов
- Blobs содержимое файлов
- Tags метки версий

## 4 Создание локального репозитория

Git создаёт папку проекта, инициализирует .git, настраивает remote и удалённые ветки.



## 5 Создание и настройка локальной ветки

Git создаёт локальную ветку (обычно main), которая отслеживает соответствующую удалённую ветку.



После clone локальная ветка и origin/main указывают на один и тот же коммит.

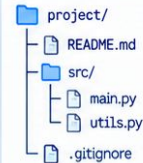
## 6 Checkout рабочей копии

Git разворачивает файлы из последнего коммита выбранной ветки в рабочую директорию.

```
# это происходит автоматически при clone
git checkout main
```

✓ В результате в папке проекта появляются обычные файлы, с которыми можно работать.

Рабочая директория



## 7 Что в итоге

- ✓ Локальный Git-репозиторий с полной историей
- ✓ Настроен remote origin
- ✓ Созданы remote-tracking ветки (origin/main и др.)
- ✓ Создана локальная ветка (обычно main)
- ✓ Файлы проекта развернуты в рабочую папку
- ✓ Можно работать без интернета (история уже локально)
- ✓ Далее можно выполнять git pull, git push и др.



## Вывод

`git clone` — это не просто скачивание файлов, а создание полноценного локального Git-репозитория с историей, ветками и настройками. После клонирования у вас есть вся необходимая информация для работы, даже без доступа к интернету.

# git fetch / git pull / git push

## 1 git fetch

Получает новую информацию из удалённого репозитория, но не меняет вашу текущую ветку. Скачивает новые commits, ветки, теги и обновляет только remote-tracking ветки (например, origin/main).

```
$ git fetch origin
```



- ✓ Скачивает новые commits, ветки, теги
- ✓ Обновляет только remote-tracking ветки (например, origin/main)
- ✓ Не трогает вашу текущую ветку (main)

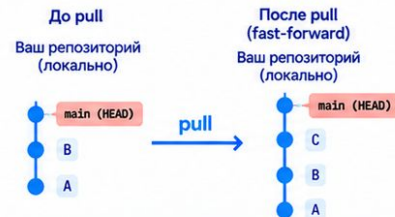
Посмотреть, что изменилось:

```
$ git log main..origin/main  
$ git diff main..origin/main
```

## 2 git pull

Это комбинация fetch + merge (или rebase). Получает изменения и пытается применить их к вашей текущей ветке.

```
$ git pull origin main  
# эквивалентно: git fetch + git merge
```



Возможные варианты результата:

- ✓ Fast-forward (если нет локальных изменений)
- ✓ Merge commit (если были локальные коммиты)
- ✓ Rebase (если настроен pull --rebase)

## 3 git push

Отправляет ваши локальные commits и refs в удалённый репозиторий. Загружает недостающие объекты (commits, trees, blobs) и обновляет ветку на удалённом репозитории.

```
$ git push origin main
```



- ✓ Передаёт недостающие объекты (commits, trees, blobs)
- ✓ Обновляет ветку на удалённом репозитории
- ✓ После успешного push локальная и удалённая ветка обычно указывают на один и тот же commit

⚠ Если на удалённом репозитории есть новые изменения, которые у вас отсутствуют, push будет отклонён. Сначала выполните git fetch (или git pull).

## 4 Сравнение команд

| Команда   | Направление    | Что делает                                                               | Меняет текущую ветку |
|-----------|----------------|--------------------------------------------------------------------------|----------------------|
| git fetch | Remote → Local | Скачивает новые commits, ветки, теги и обновляет remote-tracking ветки   | ✗ Нет                |
| git pull  | Remote → Local | Получает изменения и сразу применяет их к вашей ветке (merge или rebase) | ✓ Да                 |
| git push  | Local → Remote | Отправляет локальные commits и refs                                      | ✗ Нет                |

## 5 Схема направления данных



Локальный репозиторий  
(ваш компьютер)

## Главная мысль

- ✓ git fetch — получает изменения с удалённого репозитория, не трогая вашу текущую ветку.
- ✓ git pull — получает изменения и сразу применяет их к вашей ветке (merge или rebase).
- ✓ git push — отправляет ваши локальные commits на удалённый репозиторий.
- ✓ Если не уверены — сначала выполните git fetch, чтобы посмотреть изменения.

# .github/

## 1 Что такое .github/

.github/ — это обычная папка в репозитории, в которой хранятся настройки и конфигурация для работы с GitHub (CI/CD, шаблоны, правила и др.).



### Git

видит .github/ как обычные файлы (они версионятся).



### GitHub

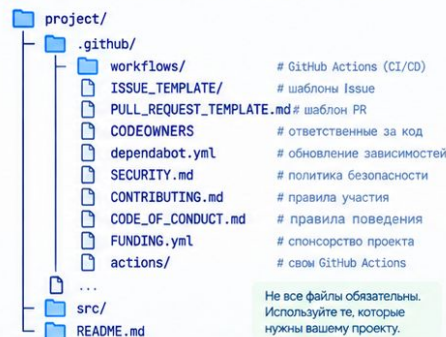
распознаёт эти файлы и использует их для автоматизации и организации работы.



.github/ коммитится в репозиторий и доступна всем участникам проекта.

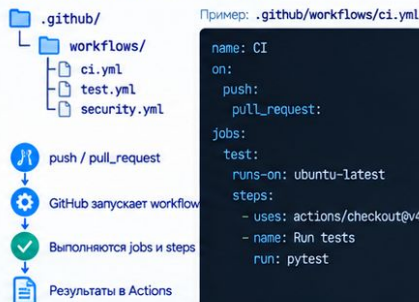
## 2 Типичная структура

Как обычно выглядит папка .github/ в проекте:



## 3 workflows/ — GitHub Actions

В этой папке лежат YAML-файлы, которые описывают автоматические процессы (CI/CD).



## 4 Шаблоны для Issues и PR

ISSUE\_TEMPLATE/ — шаблоны для создания задач.



Пользователь видит готовые формы при создании Issue.

PULL\_REQUEST\_TEMPLATE.md — шаблон для PR.

```
## Что изменено
Опишите изменения.

## Чек-лист
- [ ] Тесты проходят
- [ ] Документация обновлена
- [ ] Нет секретов в коде
```

## 5 CODEOWNERS

Определяет, кто должен проверять изменения в разных частях проекта. GitHub автоматически назначает ревьюеров.

```
.github/CODEOWNERS
# Глобальные правила
* @team/core

# Для отдельных директорий
/src/security/ @team/security
/docs/ @team/docs
```

Как это работает:

- 1 Изменяется файла
- 2 GitHub ищет правило в CODEOWNERS
- 3 Назначает указанных пользователей в ревьюеры PR

## 6 dependabot.yml

Настройка автоматического обновления зависимостей (Dependabot).

```
.github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "weekly"
```



Dependabot регулярно проверяет зависимости и создаёт Pull Request с обновлениями.

## 7 Другие полезные файлы



**SECURITY.md**  
Как сообщать об уязвимостях, поддерживаемые версии.



**CONTRIBUTING.md**  
Правила участия в разработке.



**CODE\_OF\_CONDUCT.md**  
Правила поведения в сообществе.



**FUNDING.yml**  
Настройка кнопки "Спонсировать".



**actions/**  
Собственные reusable Actions.

## 8 Как добавить .github/

Просто создайте нужные файлы, добавьте их Git и отправьте в репозиторий.

```
mkdir -p .github/workflows
touch .github/workflows/ci.yml
git add .github
git commit -m "Add Github configuration"
git push
```



## Вывод

.github/ — это централизованное место для настройки автоматизации, правил и процессов разработки на GitHub. Эти файлы делают работу команды удобнее, помогают поддерживать качество кода и автоматически запускают важные проверки.



Один репозиторий — много возможностей!

# Запреты

## 1 Зачем защищать main?

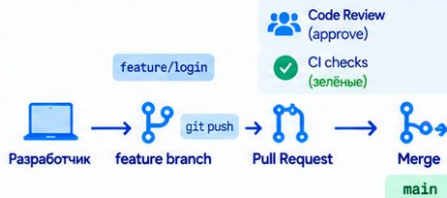
main — это стабильная версия проекта. Её нужно защищать от случайных и непроверенных изменений.

- ✓ Стабильный код в любой момент
- ✓ Меньше ошибок и инцидентов
- ✓ Обязательные проверки (CI, review)
- ✓ История изменений через Pull Request
- ✓ Прозрачный процесс разработки

❗ Прямой push в main может сломать проект для всех участников.

## 2 Как это работает?

Изменения попадают в main только через Pull Request и после выполнения всех правил.



💡 Никто (кроме администраторов) не может напрямую отправлять изменения в main. Все изменения проходят PR, проверки и ревью.

## 3 Основные правила (Branch Protection)

Типовой набор обязательных правил для main.

- ✓ **Запретить прямой push в main**  
Require a pull request before merging
- ✓ **Требовать Pull Request**  
Все изменения только через PR
- ✓ **Минимум 1 одобрение (approve)**  
Require approvals (например, 1)
- ✓ **Запретить merge при Request changes**  
Dismiss stale reviews / Require approval
- ✓ **Обязательные статус-проверки CI**  
Require status checks to pass
- ✓ **Все обсуждения должны быть resolved**  
Require conversation resolution
- ✓ **Запретить force push**  
Include administrators (запрет для всех)
- ✓ **Запретить удаление ветки**  
Prevent branch deletion
- ✓ **Требовать актуальность ветки**  
Require branches to be up to date (необязательно)
- ✓ **Требовать одобрение CODEOWNERS**  
Require review from Code Owners (для критичных частей)

## 4 Статус-проверки (CI)

Merge возможен только если все обязательные проверки прошли успешно.

|   |                                                      |            |
|---|------------------------------------------------------|------------|
| ✓ | <b>All checks have passed</b><br>4 successful checks |            |
| ✓ | build                                                | successful |
| ✓ | tests                                                | successful |
| ✓ | lint                                                 | successful |
| ✓ | security / trufflehog                                | successful |

|   |                                              |            |
|---|----------------------------------------------|------------|
| ✗ | <b>Checks have failed</b><br>1 failing check |            |
| ✓ | build                                        | successful |
| ✗ | tests                                        | failed     |
| ✓ | lint                                         | successful |
| ✓ | security / trufflehog                        | successful |

## 5 Пример конфигурации (Ruleset / Branch protection)

Настройка через GitHub UI (Settings → Rules → New ruleset) или API.

```
# Пример (упрощённый, как в ruleset)
ruleset:
- name: Protect main
  target: branch
  pattern: main
  rules:
    - type: require_pull_request
    - type: require_status_checks
    - checks: ["build", "tests", "lint"]
    - type: require_approvals
    - required_approving_review_count: 1
    - type: block_force_pushes
    - type: block_deletions
```

Где настраивается в GitHub:



## 6 Роли и исключения

Правила применяются ко всем участникам (включая администраторов, если указано).

| ✓ Обычно разрешено           | ✗ Ограничено                  |
|------------------------------|-------------------------------|
| ✓ Создавать feature branches | ✗ Прямой push в main          |
| ✓ Push в свои ветки          | ✗ Force push в main           |
| ✓ Открывать Pull Request     | ✗ Удаление main               |
| ✓ Комментировать             | ✗ Merge без одобрения         |
| ✓ Делать code review         | ✗ Merge при красном CI        |
|                              | ✗ Обход обязательных проверок |

👤 Для критичных путей (например, /src/security/) можно использовать CODEOWNERS.

## 7 Пример PR с проверками

Что происходит при создании Pull Request.



❗ Если хотя бы одно условие не выполнено — кнопка Merge будет недоступна.

## Вывод

Защищённая ветка main + обязательные проверки + code review = стабильный, безопасный и качественный код. Используйте Pull Request, а не прямой push, и доверяйте автоматические проверки CI.

✓ Хорошие правила экономят время и защищают команду!

# Pull Request

## 1 Что такое Pull Request?

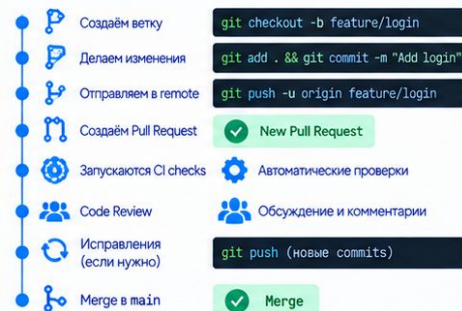
Pull Request (PR) — это запрос на объединение изменений одной ветки с другой. Это инструмент GitHub для обсуждения, проверки и совместной разработки.



- ✓ Не является частью Git (это функция GitHub)
- ✓ Позволяет проводить code review
- ✓ Запускает автоматические проверки (CI)
- ✓ Помогает делать код качественнее и безопаснее

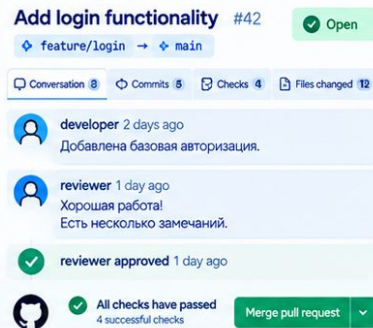
## 2 Типичный workflow с PR

Как изменение попадает из вашей ветки в main.



## 3 Что происходит в PR?

PR содержит всю важную информацию об изменении.



## 4 Шаблон для Pull Request

Файл `.github/PULL_REQUEST_TEMPLATE.md` автоматически подставляет текст при создании PR.

```
## Что изменено
Кратко опишите изменения.

## Почему это нужно
Укажите задачу или проблему (ссылка на Issue).

## Как проверялось
Опишите, как вы тестировали изменения.

## Чек-лист
- [ ] Код запускается
- [ ] Тесты проходят
- [ ] Нет секретов в коде
- [ ] Документация обновлена
- [ ] .gitignore актуален
```

## 5 Code Review в PR

Ревьюеры оставляют комментарии прямо к коду. Обсуждения должны быть закрыты перед merge.

```
12 def authenticate(username, password):
13     if password == "123456":
14 +     if check_password(password):
15         return True
```

Результаты ревью:

- ✓ Approve — можно merge
- ✓ Request changes — нужно исправить
- ✓ Comment — просто комментарий
- ✓ Все обсуждения должны быть resolved

reviewer 1 day ago  
Не храните пароли в коде. Используйте безопасную проверку.

Reply

## 6 Статус-проверки (CI)

Merge возможен только если все обязательные проверки прошли успешно.

|   |                                                                                    |                             |            |        |
|---|------------------------------------------------------------------------------------|-----------------------------|------------|--------|
| ✓ | build                                                                              | Сборка проекта              | successful | 12s    |
| ✓ | tests                                                                              | Запуск тестов               | successful | 1m 24s |
| ✓ | lint                                                                               | Проверка стиля кода         | successful | 18s    |
| ✓ | security                                                                           | Поиск секретов (TruffleHog) | successful | 45s    |
| ✗ | sast                                                                               | Статический анализ          | failed     | 2m 10s |
| ! | Если хотя бы одна обязательная проверка не прошла — кнопка Merge будет недоступна. |                             |            |        |

## 7 Типы merge

GitHub предлагает несколько способов объединения изменений.



- Создаёт отдельный merge commit
- Сохраняет всю историю
- Подходит для крупных проектов



- Все commits сжимаются в один
- Чистая история
- Удобно для учебных и небольших проектов



- Применяет commits поверх main
- Линейная история
- Подходит, если нужны отдельные commits



## Вывод

Pull Request — это не просто способ слить ветки, а полноценный процесс: код → автоматические проверки → ревью → обсуждение → merge. Используйте PR и шаблоны — это делает разработку прозрачной, безопасной и командной.



Хороший PR = качественный и надёжный проект!

# Code Review

## 1 Что такое Code Review?

Code Review — это проверка изменений в Pull Request другим участником команды. Помогает найти ошибки, улучшить качество кода и снизить риски.

- Более качественный код
- Обмен знаниями в команде
- Раннее обнаружение ошибок
- Единый стиль и архитектура

Code Review — это не критика, а совместная работа над лучшим решением.

## 2 Как проходит review?

Типичный процесс после создания Pull Request.

- Открыть Pull Request**  
Автор создаёт PR и запрашивает review
- Ревьюер смотрит изменения**  
Анализирует код, оставляет комментарии
- Обсуждение**  
Автор отвечает и вносит исправления
- Повторный review**  
Проверяются новые commits
- Approve**  
После одобрения (и успешного CI) — merge

## 3 Типы решений ревьюера

В GitHub есть три основных варианта.

- Approve** (Рекомендует к merge)
  - Код выглядит хорошо
  - Замечания учтены
  - При выполнении всех условий merge возможен
- Comment** (Просто комментарии)
  - Вопросы, предложения, обсуждение
  - Не блокирует merge
  - Можно оставить к конкретной строке или ко всему PR
- Request changes** (Требует доработки)
  - Найдены проблемы, которые нужно исправить
  - Блокирует merge (при включённой защите ветки)
  - После исправлений нужен повторный review и новое одобрение

## 4 Примеры комментариев

Комментарии могут быть общими или к конкретным строкам кода (inline comments).

```
src/auth.py
@@ -10,7 +10,7 @@ def authenticate(username, password):
10 def authenticate(username, password):
11 - if password == "123456":
11 + if check_password(password):
12     return True
```

- reviewer** 2 days ago (Request changes)  
Не храните пароли в коде. Используйте функцию `check_password()` и переменные окружения.
- author** 2 days ago (Resolve conversation)  
Исправил, спасибо!

## 5 Где смотреть результаты review?

В Pull Request на вкладке Conversation, Commits, Files changed.

### Add login functionality #42

feature/Login → main

Conversation 8 | Commits 5 | Checks 4 | Files changed 12

- reviewer** 1 day ago  
В целом хорошо, есть несколько замечаний. Смотрите комментарии ниже.

## 6 Что делать после замечаний?

Вносите исправления в ту же ветку — PR обновится автоматически.

```
# Вносим исправления в код
git add .
git commit -m "Fix comments from review"
git push
```

```
# PR автоматически обновится новыми commits
# Дополнительный PR создавать не нужно
```

- Важно:**
  - Исправления делаются в ту же ветку
  - Новые commits автоматически попадают в PR
  - После исправлений нужно дождаться нового review и одобрения

## 7 Чек-лист для ревьюера

На что стоит обращать внимание при проверке кода.

- Код решает задачу и соответствует требованиям
- Нет явных ошибок и уязвимостей
- Соблюдается стиль и стандарты проекта
- Покрыт тестами (если требуется)
- Понятные имена и структура кода
- Нет лишнего кода / комментариев
- Учтены все обсуждения из предыдущих ревью



## Вывод

Code Review — это важный этап разработки, который делает код лучше, безопаснее и понятнее. Используйте комментарии, обсуждайте решения и относитесь к ревью как к совместной работе, а не формальности.

- Хороший ревью сегодня — меньше багов завтра!

# Issues

## 1 Что такое Issue?

Issue — это задача, баг, идея или вопрос, который команда фиксирует в репозитории. Это инструмент для организации работы вокруг кода (не часть Git).

- Отслеживание задач и багов
- Обсуждение и обмен идеями
- Планирование работы
- Прозрачная история и контекст

Issue отвечает на вопрос:  
**что нужно сделать и зачем?**

## 2 Из чего состоит Issue?

Основные элементы, которые можно указать при создании задачи.

**#42 Добавить авторизацию** Open

**artem** opened 2 days ago

Необходимо реализовать регистрацию и вход пользователя в систему.

Assignee: **@artem**

Labels: **feature** **high priority**

Milestone: **v1.2**

Projects: **Backend**

## 3 Типы Issue и метки (Labels)

Метки помогают классифицировать и быстро находить нужные задачи.

### Типы задач

- bug** Ошибка в работе
- feature** Новая функциональность
- documentation** Документация
- question** Вопрос
- enhancement** Улучшение
- security** Проблемы безопасности

### Примеры приоритетов

- low** **medium** **high** **critical**

## 4 Шаблоны для Issue

Шаблоны помогают создавать качественные и полностью заполненные задачи.

```
.github/  
├── ISSUE_TEMPLATE/  
│   ├── bug.yml  
│   └── feature.yml
```

GitHub показывает готовые формы при создании Issue.

### Bug report (пример)

- ☐ Что произошло?
- ☐ Как воспроизвести?
- ☐ Что ожидалось?
- ☐ Фактический результат
- ☐ Версия приложения
- ☐ Логи / скриншоты

### Feature request (пример)

- ☒ Что нужно добавить?
- ☒ Какую проблему решает?
- ☒ Предлагаемое поведение
- ☒ Альтернативы
- ☒ Дополнительный контекст

## 5 Связь Issue → Branch → Pull Request → Close

Типичный жизненный цикл задачи.



В описании PR можно указать связь с задачей:

```
Fixes #42 # автоматически закроет issue после merge
```

## 6 Работа с Issue через CLI (GitHub CLI)

Создание, просмотр и закрытие задач из терминала.

```
# Создать issue  
gh issue create --title "Добавить авторизацию" \\  
--body "Реализовать регистрацию и вход" \\  
--label feature --assignee @artem
```

### # Посмотреть список

```
gh issue list
```

### # Посмотреть issue

```
gh issue view 42
```

### # Закрыть issue

```
gh issue close 42 --comment "Готово ✅"
```

## 7 Статусы и жизненный цикл

Основные статусы задачи в GitHub.

- Open** Задача открыта, требует решения
- In progress** В работе (можно настроить через Projects)
- In review** Реализовано, на проверке (есть PR)
- Closed** Задача выполнена или больше не актуальна



Issue можно закрыть не только кодом: например, если это дубликат, вопрос решён в документации или задача больше не актуальна.

**Вывод** Issue — это центральный инструмент для планирования и обсуждения задач. Правильно оформленные issue, метки и шаблоны делают разработку более прозрачной, помогают команде и упрощают сопровождение проекта.

Хорошие issue = понятные задачи + меньше хаоса!

# GitHub Pages

## 1 Что такое GitHub Pages?

GitHub Pages — это бесплатный хостинг для статических сайтов, который позволяет публиковать веб-сайты прямо из репозитория.

- Публикация документации
- Персональные сайты / портфолио
- Лендинги и учебные проекты
- Статические frontend-приложения
- Демо-страницы для проектов

💡 GitHub Pages берёт файлы из репозитория и делает их доступными по обычному URL.

## 2 Как это работает?

Вы пушите файлы в репозиторий, GitHub автоматически публикует их как сайт.



Примеры URL:

- Пользовательский сайт: <https://username.github.io>
  - Сайт проекта: <https://username.github.io/project-name>
- Можно использовать свой домен (например, docs.example.com)

## 3 Что можно и нельзя?

GitHub Pages предназначен для статических сайтов.

Можно ✓

- HTML, CSS, JavaScript
- Статические сайты
- Документация (например, с помощью MkDocs, Docusaurus)
- Собранные frontend-приложения (React, Vue, Vite и др.)
- Изображения, шрифты, любые статические файлы

Нельзя ✗

- Запускать backend (например, Python, Node.js)
- Использовать базы данных
- Выполнять серверный код
- Долгие фоновые задачи
- Откладывать логику на сервер

ℹ Если нужен backend — используйте другие решения (например, VPS, облачные сервисы).

## 4 Как включить GitHub Pages?

Самый простой способ — через настройки репозитория.

- Откройте репозиторий на GitHub
- Перейдите в Settings → Pages
- Выберите источник (Branch и папку)
- Сохраните — сайт будет опубликован
- После push изменения автоматически появятся на сайте

Build and deployment

Source

Deploy from a branch ▾

Branch

main ▾

/docs ▾

Save

## 5 Пример структуры проекта

Простой статический сайт в папке docs/.

```
my-website/  
├── docs/  
│   ├── index.html  
│   ├── style.css  
│   ├── script.js  
│   └── images/  
│       └── logo.png  
└── README.md
```

Если выбрать папку docs/ в настройках Pages, будет опубликовано её содержимое.

🔗 Результат:

<https://username.github.io/my-website/>

## 6 Пример простого сайта

Минимальный index.html, который можно опубликовать.

```
<!DOCTYPE html>  
<html lang="ru">  
  <head>  
    <meta charset="UTF-8">  
    <title>Мой сайт на GitHub Pages</title>  
    <link rel="stylesheet" href="style.css">  
  </head>  
  <body>  
    <h1>Привет, GitHub Pages!</h1>  
    <p>Это мой первый сайт, опубликованный из репозитория.</p>  
  </body>  
</html>
```

username.github.io/my-website/

Привет, GitHub Pages!

Это мой первый сайт, опубликованный из репозитория.



## 7 Публикация через GitHub Actions

Удобно, если нужна сборка (например, React, Vite).

```
# .github/workflows/deploy.yml  
name: Deploy to GitHub Pages  
on:  
  push:  
    branches: ( main )  
jobs:  
  deploy:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
      - uses: actions/setup-node@v4  
        with:  
          node-version: 20  
      - run: npm ci && npm run build  
      - uses: actions/upload-pages-artifact@v4  
        with:  
          path: dist  
      - uses: actions/deploy-pages@v4
```

Что происходит:

- При push запускается сборка проекта
- Собранные файлы (например, dist/) публикуются на GitHub Pages
- Сайт обновляется автоматически

✓ Подходит для React, Vue, Vite и других проектов, где нужна сборка. Сброс.



Вывод

GitHub Pages — это простой и удобный способ опубликовать статический сайт прямо из репозитория. Он отлично подходит для документации, портфолио и учебных проектов, а в связке с GitHub Actions позволяет настроить автоматическую публикацию.



Push в Git → обновлённый сайт в интернете!

# GitHub Actions

## 1 Что такое GitHub Actions?

GitHub Actions — это инструмент для автоматизации задач в репозитории. Позволяет запускать сборку, тесты, проверки и деплой при различных событиях (push, pull request и др.).

- ⚙ Автоматизация разработки
- 🔄 Интеграция с GitHub
- 📦 Готовые действия (Actions)
- 👥 Подходит для любых проектов
- ∞ Основа для CI/CD

💡 GitHub Actions выполняет инструкции из YAML-файлов в вашем репозитории.

## 2 Как это работает?

Событие в репозитории запускает workflow, который выполняется на виртуальной машине (runner) и состоит из jobs и steps.



📄 Вы описываете, что нужно сделать, а GitHub запускает это автоматически.

## 3 Структура workflow

Workflow описывается в YAML-файле в папке `.github/workflows/`.

📁 `.github/`  
└─ `workflows/`  
    └─ `ci.yml`

В репозитории может быть несколько workflows.

```
# .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [ main ]
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: python -m pip install -r requirements.txt
      - run: pytest
```

## 4 Основные элементы

Из чего состоит workflow и за что отвечает каждый элемент.

|           |                                                                                |
|-----------|--------------------------------------------------------------------------------|
| ⚡ on      | События, при которых запускается workflow (push, pull_request, schedule и др.) |
| 🔗 jobs    | Набор задач, которые могут выполняться параллельно или последовательно.        |
| 📋 runs-on | Тип runner (например, ubuntu-latest, windows-latest, self-hosted).             |
| 📄 steps   | Последовательность шагов внутри job.                                           |
| ➤ run     | Выполнение команды в терминале.                                                |
| 📦 uses    | Использование готового Action (например, actions/checkout@v4).                 |
| 🔒 secrets | Безопасное хранение секретов (пароли, токены, ключи).                          |

## 5 Пример простого CI

Проверка кода при каждом push и pull request.

```
name: Python CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      with:
        python-version: '3.11'
      - run: pip install -r requirements.txt
      - run: pytest
```

### ✅ Что происходит?

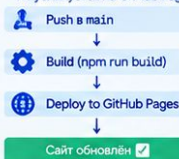
- 1 Код пушится или создается PR
- 2 Запускается workflow
- 3 Устанавливается Python
- 4 Устанавливаются зависимости
- 5 Запускаются тесты
- 6 В результате — статус в PR

## 6 Пример CI/CD

Сборка проекта, тесты и деплой на GitHub Pages.

```
name: Build and Deploy
on:
  push:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run build
      - uses: actions/upload-pages-artifact@v3
        with:
          path: ./dist
  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/deploy-pages@v4
```

✅ **Результат**  
После push в main проект автоматически собирается и публикуется на GitHub Pages.



## 7 Примеры задач для автоматизации

Что ещё можно делать с помощью GitHub Actions.

- ✅ Запуск тестов (unit, integration)
- 🛡 Статический анализ (lint, SAST)
- 📦 Сборка артефактов (Docker, бинарники)
- 🚀 Деплой (GitHub Pages, сервер, облако)
- 📄 Проверка лицензий и зависимостей
- 📢 Уведомления (Slack, Telegram, Email)
- 🕒 Запуск по расписанию (cron)
- ⚙ Автоматическое создание релизов



## Вывод

GitHub Actions позволяет автоматизировать весь процесс разработки: от проверок кода до публикации проекта. Это ключевой элемент современных CI/CD-практик, который делает разработку быстрее, безопаснее и надёжнее.

✅ Автоматизируйте рутину — сосредоточьтесь на важном!

# Бонус: TruffleHog

## 1 Что такое TruffleHog?

TruffleHog — инструмент для поиска секретов в репозитории: токенов, API keys, паролей и других чувствительных данных. Проверяет как текущие изменения, так и историю коммитов.

- API keys и токены (GitHub, AWS и др.)
- Учётные данные (пароли, логины)
- Private keys и сертификаты
- Конфигурационные файлы (.env, .pem и др.)
- Поиск в текущих изменениях и в истории

💡 TruffleHog — не SAST. Он не ищет уязвимости в коде, а находит секреты.

## 2 Как это работает в CI?

При каждом push или создании Pull Request GitHub Actions запускает TruffleHog, который сканирует код и историю.



### Что проверяется?

- Изменённые файлы в Pull Request
- Вся история репозитория (при необходимости)
- Строки, соответствующие известным паттернам секретов
- Оptionальная проверка валидности найденных секретов

## 3 Пример workflow (GitHub Actions)

Простой пример .github/workflows/secret-scan.yml с запуском TruffleHog.

```
name: Secret Scan
on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  trufflehog:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0 # чтобы сканировать историю

      - name: Run TruffleHog
        uses: trufflesecurity/trufflehog@main
        with:
          path: ./
          extra_args: --severity HIGH,CRITICAL
```

## 4 Примеры находок

TruffleHog может найти различные типы секретов. Примеры (упрощённо):

```
$ trufflehog git .
Found potential secret:
  Detector: GitHub Token
  Location: src/config.py:12
  Match: ghp_XXXXXXXXXXXXXXXXXXXX

---
Found potential secret:
  Detector: AWS Access Key
  Location: .env:3
  Match: AKIAXXXXXXXXXXXXXXXX

---
Found potential secret:
  Detector: Private Key
  Location: keys/id_rsa:1
  Match: -----BEGIN RSA PRIVATE KEY-----
```

❗ Даже если файл позже удалён, секрет может остаться в истории Git и быть найден.

## 5 Пример реальной ситуации

Разработчик случайно добавил токен в код:

```
10 # config.py
11 GH_TOKEN = "ghp_XXXXXXXXXXXX"
12 print("Start app...")
13
```



### Результат в CI:

- TruffleHog находит токен
- Job падает (failed)
- Merge в main заблокирован
- Токен нужно перевыпустить

## 6 Как исправить?

- Удалить секрет из кода
- Если секрет уже был в репозитории — удалить его из истории (например, BFG)
- Перевыпустить скомпрометированный токен
- Проверить, что проблема больше не повторяется (запустить CI ещё раз)
- Добавить .gitignore, чтобы подобные файлы не попадали в репозиторий

## 7 Где использовать в пайплайне?

TruffleHog обычно запускают на ранних этапах CI:



✅ Лучше находить секреты как можно раньше — это дешевле, быстро и предотвращает утечки данных.



## Вывод

TruffleHog в CI — простой и эффективный первый шаг к безопасной разработке. Он помогает не допустить утечку секретов в репозиторий и делает процесс разработки безопаснее.



Безопасность начинается с небольших шагов!

